

71.pdf

 University of Engineering & Management

Document Details

Submission ID

trn:oid:::3618:103124130

Submission Date

Jul 1, 2025, 2:33 PM GMT+5:30

Download Date

Jul 5, 2025, 12:02 AM GMT+5:30

File Name

71.pdf

File Size

2.8 MB

24 Pages

8,719 Words

43,951 Characters

36% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **107** Not Cited or Quoted 28%
Matches with neither in-text citation nor quotation marks
-  **14** Missing Quotations 5%
Matches that are still very similar to source material
-  **8** Missing Citation 3%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 27%  Internet sources
- 30%  Publications
- 13%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 107** Not Cited or Quoted 28%
Matches with neither in-text citation nor quotation marks
- 14** Missing Quotations 5%
Matches that are still very similar to source material
- 8** Missing Citation 3%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 27% Internet sources
- 30% Publications
- 13% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	journals.pan.pl	20%
2	Publication	Mariusz Kaleta, Tomasz Śliwiński. "Neural-Driven Constructive Heuristic for 2D Ro...	11%
3	Submitted works	Universidade do Porto on 2022-03-01	<1%
4	Internet	arxiv.org	<1%
5	Internet	www.ime.usp.br	<1%
6	Publication	Y. Zhu, M. Ahuja. "On job scheduling on a hypercube", IEEE Transactions on Parall...	<1%
7	Publication	Jahangir Hossain, Nagham Saeed, Rampelli Manojkumar, Mousa Marzband, Khale...	<1%
8	Internet	ijet.pl	<1%
9	Internet	www.cs.put.poznan.pl	<1%
10	Publication	Fatih Burak Akçay, Maxence Delorme. "Solving the parallel processor scheduling ...	<1%

11	Publication	"Autonomous Search", Springer Science and Business Media LLC, 2012	<1%
12	Internet	ipfs.io	<1%
13	Publication	Jeffrey F. Queißer, Jochen J. Steil. "Bootstrapping of Parameterized Skills Through ...	<1%
14	Internet	smartech.gatech.edu	<1%
15	Publication	Hao Zhang, Shaowen Yao, Shenghui Zhang, Jiewu Leng, Lijun Wei, Qiang Liu. "A bl...	<1%
16	Publication	Ioana Sitaru, Madalina Raschip. "Algorithm Selection for Combinatorial Packing P...	<1%
17	Publication	Kun He, Yan Jin, Wenqi Huang. "Heuristics for two-dimensional strip packing prob...	<1%
18	Publication	Xiangjing Lai, Lei Wang, Jin-Kao Hao, Qinghua Wu. "An effective evolutionary algo...	<1%
19	Internet	core.ac.uk	<1%
20	Publication	Alvarez-Valdes, R.. "Reactive GRASP for the strip-packing problem", Computers an...	<1%
21	Internet	ieeexplore.ieee.org	<1%
22	Internet	iris.poliba.it	<1%
23	Internet	link.springer.com	<1%
24	Internet	nottingham-repository.worktribe.com	<1%

25	Internet	publications.polymtl.ca	<1%
26	Publication	Engineering Computations, Volume 30, Issue 4 (2013-05-27)	<1%
27	Publication	Jona Beysens, Martin Sénéclauze, Philippe Dallemagne, Siavash Bigdeli. "Early Exi...	<1%
28	Publication	Lecture Notes in Computer Science, 2003.	<1%
29	Publication	Oliveira, oscar Antonio Maia de. "Efficient Heuristics for Two-Dimensional Cutting...	<1%
30	Publication	Xu Zhao, Qianjun Lin, Hao Yu. "A Co-Scheduling Problem of Ship Lift and Ship Loc...	<1%
31	Internet	repositorium.uminho.pt	<1%
32	Internet	www.ncbi.nlm.nih.gov	<1%
33	Submitted works	Chiang Mai University on 2020-06-11	<1%
34	Publication	Qian Hu, Wenbin Zhu, Hu Qin, Andrew Lim. "A branch-and-price algorithm for the...	<1%
35	Publication	"Operational Research", Springer Science and Business Media LLC, 2018	<1%
36	Submitted works	University of Stellenbosch, South Africa on 2018-10-12	<1%

Improving Neural-Based Heuristics for 2D Strip Packing through Local Search in Heuristic Space

Mariusz Kaleta and Tomasz Śliwiński

Institute of Control and Computational Engineering,
Warsaw University of Technology, Warsaw, Poland
{mariusz.kaleta}{tomasz.sliwinski}@pw.edu.pl

Abstract. This paper presents a novel approach to the two-dimensional orthogonal rectangular strip packing problem (2D-SPP), focusing on arranging rectangular items on a strip of fixed width to minimize the total height. We extend the recently proposed novel neural-based constructive heuristic by integrating a local search strategy that explores the space of heuristics, encoded as neural network weights, rather than the solution space. Utilizing the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) for both training and problem-specific refinement, our method dynamically learns and optimizes placement decisions. We introduce benchmarking datasets inspired by real-world fast-moving consumer goods (FMCG) logistics, demonstrating the approach's efficacy. Experimental results show that our refined heuristic (RCH) significantly outperforms traditional heuristics like MaxRects and Skyline, achieving up to tenfold reductions in wasted space compared to constraint programming for larger, heterogeneous instances. The proposed method offers scalability and adaptability, making it particularly suited for practical applications in logistics and manufacturing.

Keywords: Two-Dimensional Strip Packing, Neural-Based Heuristic, Local Search, CMA-ES, Algorithm Selection.

1 Introduction

1.1 Background

We consider the two-dimensional orthogonal rectangular strip packing problem (2D-SPP), which involves arranging a set of rectangular items on a strip of fixed width W and unbounded (infinite) length. Reflecting its practical relevance, we assume that item sizes exhibit a certain repeatability—that is, the item set can be grouped into subsets of identical dimensions. This assumption holds in many real-world applications, particularly in fast-moving consumer goods (FMCG) logistics, where products such as bottled water, milk, and yogurt are packed in standardized units (e.g., 6-bottle packs, 12-box crates, 20-tray bundles).

Let $t \in \mathbf{T}$ denote an item type, and let d_t represent the number of items of that type t to be packed. The full set of items is then defined as $\mathbf{R} =$

2 M. Kaleta et al.

$\{r_1^t, r_2^t, \dots, r_{d_t}^t\}$ for all $t \in \mathbf{T}$. Each item $j \in \mathbf{R}$ has width $w_j > 0$ and height $h_j > 0$. The goal is to pack all items into the strip while minimizing the total height of the resulting packed layout.

We assume orthogonal packing, where each rectangle must be aligned with the strip axes—i.e., all edges are parallel to the edges of the strip. Items may be rotated by 90 degrees. Every item must lie entirely within the strip boundaries, and no overlap is permitted.

The 2D-SPP arises in many practical domains, including cutting materials in the wood, paper, and textile industries, placing components on semiconductor layers, parallel task scheduling in high-performance computing, and logistics and transportation loading problems [1]. The problem is NP-hard, and is commonly approached using heuristics or metaheuristics [2].

Constructive heuristics are popular due to their simplicity and speed, but they typically lack generalization capabilities. Their performance is highly instance-dependent, with no single rule performing best across all problem types. On the other hand, metaheuristics (e.g., genetic algorithms, simulated annealing, variable neighborhood search) are more flexible and capable of adapting to a wider variety of instances. However, they tend to explore the solution space from scratch for every new problem, failing to leverage prior experience or structural similarity between instances.

To address these limitations, a neural-based constructive heuristic was recently proposed in [3]. This method learns to make effective packing decisions and can dynamically adapt to the current packing state—overcoming the static behavior typical of traditional heuristics.

In this paper, we build upon the neural-based constructive approach introduced in [3]. We extend its capabilities by integrating a local search mechanism, inspired by metaheuristic strategies. Crucially, our approach does not search in the space of solutions (as is common in metaheuristics), but instead explores the space of heuristics, represented by different configurations of neural network parameters. Once a general-purpose heuristic is learned from a representative dataset, we refine it for specific instances using local search in the heuristic space, effectively adapting the strategy without discarding previously learned knowledge.

1.2 Related literature

Heuristic approaches to the two-dimensional strip packing problem focus on item selection and placement. Items are often pre-sorted by size, but advanced methods like Q-learning are also used for predicting packing sequences [4]. Most studies emphasize placement, using greedy heuristics with scoring functions [5]. Constructive heuristics select from the set of candidate placements using rules like Bottom-Left, Best-Fit [6, 7], or MaxRects [8]. The set of candidates can be generated with different techniques. The Skyline algorithm tracks the top envelope [9] to select the best placement. A similar envelope-based approach was introduced by Martello, Pisinger, and Vigo, who defined Corner Points as

1. INTRODUCTION 3

non-dominated locations where an item can be placed [10]. Extreme Points, an extension of this idea, further exploits free space within the current packing [11].

Besides constructive heuristics, the second mainstream approach is local search. Metaheuristics, such as tabu search, simulated annealing, and genetic algorithms, improve solutions through the process of searching local neighborhoods [1]. However, selecting the best heuristic or metaheuristic remains challenging, a case of the Algorithm Selection Problem [12]. Some attempts try to address that issue. Genetic algorithms and data mining are used to predict optimal heuristics [13, 14], while hyper-heuristics optimize heuristic combinations [15]. Piechowiak et al. proposed a framework of algorithm portfolios that chooses the best algorithm portfolio minimizing the cost of solving under such portfolio [16].

Recently, machine learning has emerged as a promising tool for packing problems. One approach is to use reinforcement learning (RL) for pixel-based strip representations, though it is limited by the problem size [17]. Simple approaches are based on a neural network that selects an action from a predefined set of decisions. However, in general, RL struggles with varying decision sets in combinatorial optimization [18]. Alternatively, machine learning guides heuristic selection. Fang et al. proposed a pointer network with an encoder-decoder structure to optimize packing sequences, combined with MaxRects-BL for placement [19]. Xu et al. introduced a dual graph neural network, where one network selects the next item to pack, and the other encodes free space geometry [20]. Kai Zhu et al. used a neural network and reinforcement learning as a "scorer", evaluating placement candidates [21]. Supervised learning classifies instances to recommend heuristics [22].

1.3 Research gap and contribution

While some attempts have been made to address the algorithm selection problem in the context of the two-dimensional strip packing problem (2D-SPP), work [3] was among the first to explore optimization in the space of constructive heuristics. Rather than statically selecting a predefined heuristic, we proposed a method that learns to score alternative placements dynamically, based on the current state of the partial packing.

Inspired by local search techniques commonly used in solution-space optimization, we propose a similar strategy—this time applied to the space of algorithms. To the best of our knowledge, no previous work has explored local search in the space of algorithms for 2D-SPP. This forms the basis of our contribution, which can be summarized as follows:

- We introduce a local search method that operates in the space of constructive heuristics, represented by the weights of a neural network. This extends the previously proposed neural-based framework for 2D-SPP. We demonstrate that this approach outperforms traditional constructive heuristics and constraint programming in terms of packing efficiency.

4 M. Kaleta et al.

- We show that it is possible not only to learn heuristics tailored to specific datasets, but also to further refine those heuristics for individual problem instances, improving performance through local search in the weight space.
- We present new benchmarking datasets based on real-world item dimensions from the FMCG industry. Our results show that the proposed approach is particularly effective in such practical scenarios.

The proposed neural-based constructive heuristic, combined with local search, directly addresses the algorithm selection problem. The machine learning component enables the creation of more adaptive and efficient heuristics, while the local refinement mechanism significantly enhances their performance in instance-specific settings.

1.4 Paper organisation

The reminder of this paper is structured as follows. Section 2 provides a detailed description of our neural-driven constructive heuristic. Section 3 introduces the local search component of our approach. Then, in section 4, we present numerical experiments, evaluating our approach against standard heuristics and constraint programming. Section 5 summarizes our findings.

2 Neural-based constructive heuristic

2.1 General idea

A typical constructive heuristic solves the packing problem iteratively by placing one item at a time. Recently, a framework for neural-based constructive heuristics that operates in the space of all corner-point-based heuristics has been proposed [3]. These corner points correspond to bottom- and left-aligned, non-dominated locations derived from the top envelope of items already placed on the strip [10].

Within this framework, at each iteration, a neural network $N_{\bar{w}}$ with parameters $\bar{w}$ scores all feasible placement candidates, and the candidate with the highest score is selected. Figure 1 shows the algorithm flowchart, which follows these main steps:

1. **Start:** The algorithm begins with the item set $\mathbf{R}$ to be packed and the trained neural network weights $\hat{w}$.
2. **Initialize:** Set $k = 1$. Initialize $\mathbf{R}_k$, the set of items remaining to be packed at iteration k , and the partial solution s_k , initially an empty strip.
3. **Prepare Candidates:** Generate the set of all feasible placements at iteration k , denoted $Y_k = Y(\mathbf{R}_k, s_k)$. This includes all valid placements of items in $\mathbf{R}_k$ at every corner point of the current solution s_k , both in their original orientation and rotated by 90 degrees.
4. **Select a Decision:** Identify the decision $\hat{y}_k \in Y_k$ that maximizes the neural network's score:

$$\hat{y}_k = \arg \max_{y \in Y_k} N_{\bar{w}}(p(s_k, y))$$

2. NEURAL-BASED CONSTRUCTIVE HEURISTIC 5

Here, the function $p(s_k, y)$ extracts a set of features from the current partial solution s_k and the candidate decision y , which are then fed into the neural network $N_{\hat{w}}(p(s_k, y))$ to compute a placement score of decision y assuming current state s_k .

5. **Update Solution:** Apply the decision $\hat{y}_k$ to obtain the new partial solution:

$$s_{k+1} = T(s_k, \hat{y}_k)$$

where $T(s_k, \hat{y}_k)$ is a transition function that places the item according to $\hat{y}_k$ and updates the strip layout. The placed item $r(\hat{y}_k)$ is removed from the remaining item set.

6. **Stopping Condition:** If $\mathbf{R}_{k+1}$ is not empty, increment k and return to Step 3.
7. **End:** Once all items have been placed, s_k represents the final solution. The total height of the packing is given by $H(s_k)$.

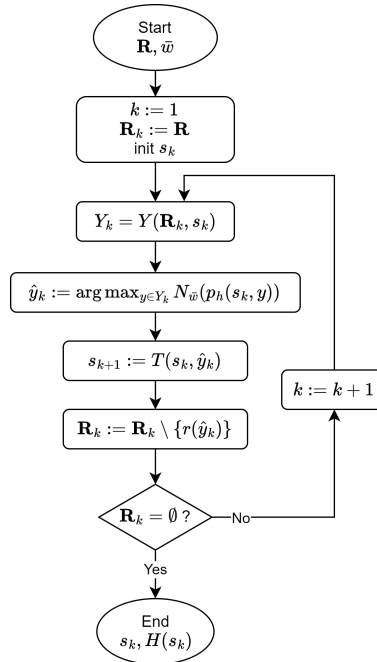


Fig. 1. General view of the neural-driven construction heuristic

It is important to note that, once trained, the neural network allows the heuristic to construct a solution in a single forward pass, without explicitly searching the solution space. This is in contrast to traditional metaheuristics, which rely on iterative exploration of multiple solution variants. Our method, therefore, offers a much more computationally efficient alternative.

6 M. Kaleta et al.

2.2 Neural network architecture

We use a small-scale and simple feed-forward neural network for $N_{\hat{w}}$ illustrated in Figure 2. The network has a simple four-layer feed forward architecture with 24 inputs and one output. The hidden layer sizes are 32 and 16, respectively. The hidden layers and the output are equipped with bias values. The hidden layers utilize an approximate $\tanh$ activation function. The total number of network parameters (weights) is 1,345. Initially (before training), all weights are set to 0.

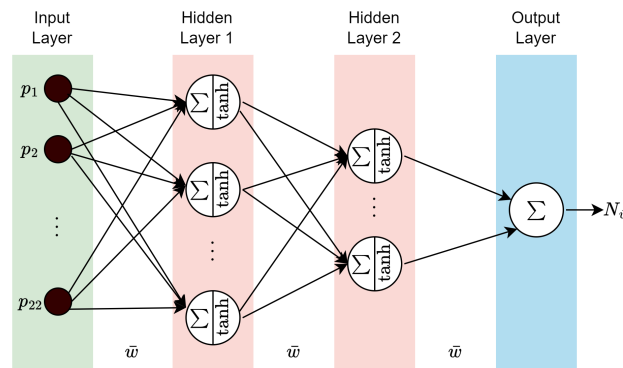


Fig. 2. Architecture of network $N_{\hat{w}}$

The input to the neural network is a set of features defined by the function $p(s_k, y)$. The selection of 24 properties resulted from expert knowledge of packaging problems and a series of experiments with different property configurations. For item j and given corner point and item's rotation, the vector of properties consists of the following (for more details we refer to [3]):

- I. Information about the item: width w_j , height h_j , and area
- II. Information about the remaining items $\mathbf{R}_k \setminus \{j\}$: number and areas of the remaining items, number and area of items of the same type as item j .
- III. Information about the state after the placement:

1D view of the strip state.

Evenly discretized top view on packed items. More precisely, the eight-long element vector containing distances between strip's height before placing j and the non-dominated envelope of packed items after placing j .

Horizontal/vertical mismatch.

This property quantifies how well the envelope of the new item j is aligned with adjacent items horizontally/vertically.

Wasted space.

The total space wasted after placing j that cannot be used in the corner points approach.

2. NEURAL-BASED CONSTRUCTIVE HEURISTIC

7

Horizontal/vertical size fit.

Properties measure a slack if the space to the right (horizontal) or above (vertical, limited by the current height of the strip) would be filled in with more elements of type j .

Horizontal/vertical distance.

Distance between item j and strip's right wall, and current strip's height.

Horizontal and vertical positions.

Similar to horizontal and vertical distance, but in relation to the bottom and left sides of the new item j rather than to the top and right sides of the new item.

Values of each property are scaled. For properties that may have outliers, including: remaining area, number of remaining items, 1D view of the strip state, horizontal and vertical mismatch, wasted space, vertical distance, and position, we apply a hyperbolic tangent function to fit values to the $[-1, 1]$ range.

We used the approximate $\tanh$ function to speed up the inference, as the original is very slow, and we do not need its full precision. In the range $[-2.779, 2.779]$ the function is approximated by the fourth degree polynomial, and outside of this range it is approximated by the linear function with slope 0.01 (see the plot on Figure 3). The range was chosen so, that the function's first derivative is continuous. Based on our preliminary experiments, the linear non-constant function defined outside of this range speeds up the learning rate. The $\tanh(x)$ piecewise approximation was formulated as follows:

$$\tanh(x) \approx \begin{cases} (x + 2.779) \cdot 0.01 - 0.998, & \text{for } x < -2.779 \\ -1 + \left(\frac{x+3.5}{3.5}\right)^4, & \text{for } -2.779 \leq x < 0 \\ 1 - \left(\frac{x-3.5}{3.5}\right)^4, & \text{for } 0 \leq x < 2.779 \\ (x - 2.779) \cdot 0.01 + 0.998, & \text{for } 2.779 \leq x \end{cases}$$

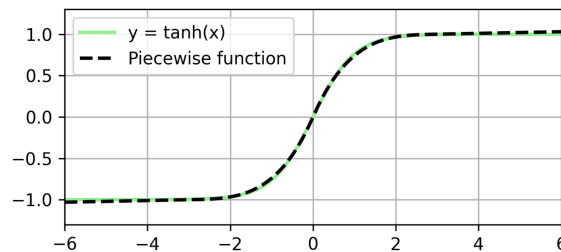


Fig. 3. Piecewise approximation of the $\tanh(x)$

2.3 Network training with black-box optimization

Since the neural network is embedded within a combinatorial construction heuristic, and there is no explicit way to determine the correct network outputs, typical

8 M. Kaleta et al.

backpropagation learning schemes cannot be applied. However, black-box optimization methods for neural network training have gained significant attention in the machine learning community [23–25].

We apply the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) black-box optimization of neural network's parameters $\bar{w}$ [26]. The general training process is illustrated in Figure 4. The algorithm iteratively processes the population of individuals, $\mathbf{N}$, where individual $n \in \mathbf{N}$ represents a set of neural network weights $\bar{w}_n$ and, consequently, a distinct constructive heuristic. Each individual from the population of neural networks is evaluated through the execution of the constructive heuristic in which the network is embedded as a scoring function. An average of the strip's height (or equivalent fill factor) over each data instance is a measure of the performance of an individual. CMA-ES searches the space of weights $[\bar{w}_n], n \in \mathbf{N}$. It perceives the constructive heuristic as a black box that delivers measures, which CMA-ES utilizes, along with estimations of the mean and covariance matrix, to generate an offspring population and update the distribution parameters.

To reduce the computational cost of evaluating the black-box function, we use a batch of 100 randomly selected problem instances from 500,000 in each CMA-ES iteration. Every 10th iteration, the best solution found in that iteration is evaluated against a predefined validation set of 10,000 problems. The best-performing solution on the validation set so far is stored as the final solution $\hat{w}$ of the algorithm.

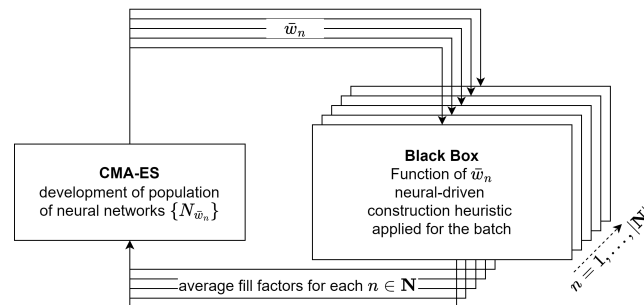


Fig. 4. Black-box optimization with CMA-ES

3 Refining the learned heuristic

The neural-driven constructive heuristic described earlier performs a single run of the algorithm, incrementally incorporating new items into a growing partial solution. Unlike traditional approaches that explore the solution space by iterating over multiple complete packings to identify the optimal one, this method avoids

3. REFINING THE LEARNED HEURISTIC 9

broad search procedures. Instead, it concentrates its exploration within each individual decision step, where the neural network evaluates multiple candidate placements and selects the most favorable one given the current packing state. This design prioritizes computational efficiency, making it highly effective—yet it leaves room for further improvement, particularly when dealing with NP-hard problems. A key characteristic of such problems is the difficulty in predicting which intermediate choices lead to the best final outcome.

Conventional methods for addressing these challenges often rely on meta-heuristics to systematically search the solution space. Local search techniques typically start with an initial solution (or a small pool of solutions), which is then iteratively refined by exploring its neighborhood. However, these methods present significant challenges for practitioners. One major difficulty lies in translating real-world combinatorial problems into representations suitable for a specific meta-heuristic, often requiring intricate adjustments to maintain solution feasibility. Furthermore, navigating large and complex search spaces raises concerns around scalability. Another limitation is that traditional meta-heuristics are restarted for each new problem instance, even when the instances are similar, and typically embed expert knowledge only at the initialization stage.

In this work, we propose a novel meta-heuristic paradigm that approaches the problem differently: instead of searching the space of solutions, we search the space of heuristics. More specifically, we explore the landscape of neural-based constructive heuristics, leveraging knowledge gained from previously solved problems—under the assumption that these problems share some structural similarities.

Consider the neural network embedded within our constructive heuristic, with parameters $\bar{w}$ trained on a set of representative problem instances. When applied to a new instance—whether previously seen or not—the heuristic uses the network to score candidate placements at each step and selects the one with the highest score. Often, several candidates receive similarly high scores, reflecting their historical effectiveness in producing strong final packings. This observation suggests an opportunity: by considering a range of high-scoring alternatives at each step, we may discover improved solutions through variations that follow proven trajectories.

One way to harness this potential is to evaluate not only the top-scoring decision, but a group of leading candidates. Several strategies are possible. For instance, we could randomly select among the top decisions, with probabilities proportional to their scores. Alternatively, a beam search strategy could be used to track multiple promising partial solutions (with a fixed beam width), expanding them based on score-weighted selection. For related approaches, see [27–29].

Our method, however, follows a different path. It is based on the insight that small perturbations in the neural network's weights can change the ranking of top decisions at each step. The closer the scores of the leading candidates, the easier it is to shift toward alternative packing trajectories by slightly adjusting the weights. In this sense, modifying the weights allows us to traverse a landscape of

10 M. Kaleta et al.

related heuristics—where each configuration of weights defines a unique neural-based heuristic.

The central challenge becomes: how to methodically guide this exploration of heuristic space to identify higher-performing configurations? To this end, we repurpose the CMA-ES algorithm—originally used to train the neural network on a general set of instances—for local refinement on a specific problem instance. The overall process is illustrated in Figure 5.

Initially, the neural network is trained using CMA-ES to optimize performance across a large, diverse set of problem instances. For this phase to be successful, the training set and resulting heuristics must share underlying structural characteristics. The best-performing parameter vector from this stage, denoted as $\hat{w}$, defines a general-purpose heuristic capable of producing good solutions in a single pass for new, similar instances.

To further refine this heuristic for a specific instance, we then re-apply CMA-ES—this time with a narrow focus on a particular problem instance. Starting with $\bar{v} := \hat{w}$, the algorithm performs a local evolutionary search in the weight space $\bar{v}_n$, seeking the configuration that yields the best packing for the single target instance. This second optimization step is highly efficient: it builds upon a pre-trained model and uses a compact, focused search to adapt the heuristic further—resulting in substantial performance gains even under tight computational budgets.

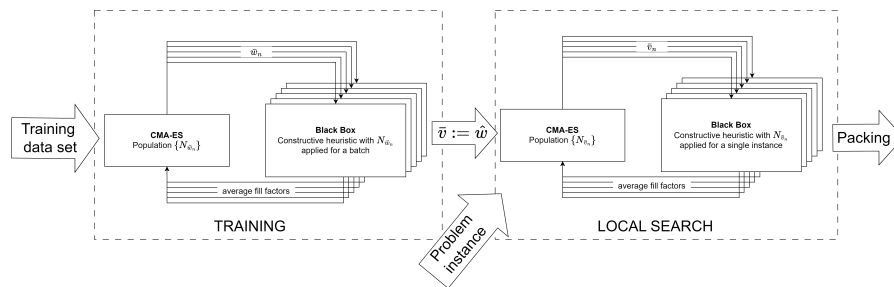


Fig. 5. Neural-based constructive heuristics with local search

4 Computational experiments

4.1 Datasets generation

We employed two distinct methods to create test problem instances. The first method utilizes the 2DCPackGen generator, tailored for two-dimensional rectangular cutting and packing challenges [30]. The second method emulates real-world logistics scenarios, using predefined strip width and a fixed set of item sizes.

4. COMPUTATIONAL EXPERIMENTS 11

In this paper, we denote these datasets as 2DCPackGen and logistic datasets, respectively.

For each generation method, problem type, and size, we produced 510,000 training instances, allocated as follows:

- 500,000 instances dedicated to neural network training.
- 10,000 instances set aside for validation, used to assess and select the top-performing neural network during training iterations.

To ensure a fair comparison between the neural-driven heuristic and the benchmark heuristics (MaxRects and Skyline), we generated an additional 100 instances for each combination of problem type and size. These instances were generated using a different random seed to keep them distinct from the training problems, and they were solely used for performance evaluation.

2DCPackGen-generated datasets. We leveraged 2DCPackGen, a tool for generating two-dimensional rectangular cutting and packing problems [30], to produce various problem instances. Adhering to the classification by Wäscher et al. [31], we focused on the *Open Dimension Problem/S* (ODP/S) type.

Each problem instance is represented by the tuple:

$$(W, w_t, l_t, d_t)$$

where:

- $t \in \mathbf{T}$ represents the set of item types,
- W is the strip's width,
- w_t and l_t denote the width and length of item type t ,
- d_t indicates the demand (number of items of type t).

Table 1 lists the relevant parameters for the ODP/S problem type used by 2DCPackGen, omitting parameters irrelevant to strip packing.

The strip width was standardized at $W = 1000$ (parameter #4). We evaluated three different counts of item types (d_t) (parameter #8): 5, 10, and 15 item types. Item dimensions (w_t, l_t) for type t were randomly generated as integers within $[100, 500]$ (parameter #5), adhering to one of four predefined shape and size profiles:

- Small and square (ID=1),
- Big and square (ID=6),
- Long and narrow (ID=2),
- Mixed sizes, including small and square, short and tall, long and narrow, or big and square (ID=16),

where the ID aligns with 2DCPackGenitem's item shape classification (parameter #6).

Item demand was modeled as a uniform random variable, determined by the range specified in parameter #9.

We explored problems with varying numbers of item types (parameter #8), different demand generation ranges (parameter #9), and diverse size and shape characteristics for small item types (parameter #6).

Table 1. Parameters of the 2DCPackGen problem instances generator

#	Parameter name	Value ([min, max])
1	Number of Dimensions:	2
2	Integer seed:	100 or 10000
3	Number of instances:	100 or 510000
4	Width of the strip:	[1000, 1000]
5	Items' size dimensions:	[100, 500]
6	ID of the size and shape characteristic of items:	1, 6, 2 or 16
7	Number of bins (strips):	[1, 1]
8	Number of different item types:	[5, 5], [10, 10] or [15, 15]
9	Number for item type demand:	[1, 5], [1, 10] or [1, 20]
10	ID of the characteristic for the item type demand:	5

Real-life logistic data sets. In pallet-based packing, item sizes are typically standardized—e.g., yogurt trays, water packs, or canned goods maintain consistent dimensions across producers. These sizes are designed to align with standard pallet dimensions, such as the Euro pallet (1200 × 800 mm), to optimize space usage. A single layer of identical items ideally maximizes pallet coverage, often achieved through grid-based patterns where item dimensions are divisors of the pallet's dimensions. While other patterns are possible, they are less common in practice.

Drawing from real-world FMCG industry dimensions, we defined a set of item sizes, categorized as small, medium, and large. Table 2 details the dimensions used in our experiments. Except for (300, 240) and (250, 200), all dimensions support grid-based layer formation on a standard Euro 1 pallet. The exceptions allow single-layer packing but require alternative patterns.

We classified items on their larger dimension, using the following size ranges:

- Small: 100–160 mm
- Medium: 171–250 mm
- Large: 266–600 mm

Additionally, we introduced a fourth category (Mixed), which includes all Small, Medium, and Large item sizes.

Consistent with prior setups, we generated problems with 5, 10, or 15 distinct item types. Demand for each type was a uniformly random integer from the ranges 1–5, 1–10, or 1–20.

To generate random test instances for a given item dimension category, number of item types, and demand range, we first sampled n unique dimensions from the category. Each dimension was then assigned a demand as a uniformly distributed random integer within the specified range. The strip width was consistently set to 1200 mm, matching the standard Euro pallet width.

Table 3 summarizes key statistics for the generated problem instances (both 2DCPackGen and logistic types), including the average number of items and a

4. COMPUTATIONAL EXPERIMENTS 13

Table 2. Item dimensions (mm) for the real-life logistic problems

Small	Medium	Large
(100, 100)	(171, 100)	(266, 109)
(109, 100)	(171, 114)	(266, 120)
(114, 100)	(171, 133)	(266, 133)
(114, 109)	(171, 160)	(266, 150)
(120, 100)	(200, 100)	(266, 171)
(120, 114)	(200, 109)	(266, 200)
(133, 100)	(200, 114)	(266, 240)
(133, 109)	(200, 120)	(300, 133)
(133, 114)	(200, 133)	(300, 160)
(133, 120)	(200, 150)	(300, 200)
(133, 133)	(200, 160)	(300, 240)
(150, 100)	(200, 171)	(300, 266)
(150, 114)	(200, 200)	(400, 160)
(150, 133)	(240, 100)	(400, 171)
(160, 100)	(240, 114)	(400, 200)
(160, 109)	(240, 133)	(400, 240)
(160, 120)	(240, 160)	(400, 266)
(160, 133)	(240, 200)	(400, 300)
(160, 150)	(250, 200)	(400, 400)
		(600, 266)
		(600, 400)

heterogeneity measure (calculated as the number of item types divided by the average number of items per instance).

Since the proposed approach relies heavily on data, we carefully selected data generation parameters for the experiments, specifically the strip width to item size proportions and item aspect ratios. The experiments primarily target applications in the FMCG industry and may not encompass all possible scenarios. Both datasets cover a similar range of strip width to item size proportions, varying from 2 to 10 for 2DPackGen and from 2 to 12 for logistic instances. The item aspect ratios range from 1 to 5 for 2DPackGen instances and from 1 to 6 for logistic instances. Although we anticipate that broader ranges could still function within the proposed framework, further research is needed to determine the limits of applicability.

4.2 Test bed

We benchmarked our approach against the best solutions obtained from well-known single-pass heuristic families: MaxRects and Skyline [8], as well as from Constraint Programming models (CP).

MaxRects and Skyline heuristic families' main principle is sorting the items, then positioning them within the strip. For item sorting, we consider descending order based on the following attributes:

14 M. Kaleta et al.

Table 3. Statistics of problem instances

num. of item types	item type demand	avg. num. of items	heterogeneity measure
5	1–5	15.0	0.33
	1–10	27.5	0.18
	1–20	52.5	0.09
10	1–5	30.0	0.33
	1–10	55.0	0.18
	1–20	105.0	0.09
15	1–5	45.0	0.33
	1–10	82.5	0.18
	1–20	157.5	0.09

- Item area,
- Item perimeter,
- Absolute difference in side lengths,
- Shorter side length,
- Longer side length,
- Ratio of side lengths.

For placement selection, we tested all standard criteria for MaxRects and Skyline, yielding ten heuristics as described in [8]:

- MaxRects-Based Heuristics:
 - Maximal Rectangles Bottom-Left (MaxRectsBl),
 - Maximal Rectangles Best Area Fit (MaxRectsBaf),
 - Maximal Rectangles Best Short Side Fit (MaxRectsBssf),
 - Maximal Rectangles Best Long Side Fit (MaxRectsBlsf),
- Skyline-Based Heuristics:
 - Skyline Bottom-Left (SkylineBl),
 - Skyline Bottom-Left Waste Map Improvement (SkylineBlWm),
 - Skyline Min Waste Fit (SkylineMwf),
 - Skyline Min Waste Fit with low profile (SkylineMwfl),
 - Skyline Min Waste Fit with Waste Map Improvement (SkylineMwfWm),
 - Skyline Min Waste Fit with low profile and with Waste Map Improvement (SkylineMwflWm).

Additionally, we test a global approach (as described in [8]) where items are not pre-sorted. Instead, the placement score determines the next item to place. In this strategy, we evaluate placement scores and choose the best combination of item and placement position. For instance, in the MaxRects-BSSF variant, the selected item and position minimize the shorter leftover side, defined as $\min_{(f,i)} (\min(w_f - w_i, h_f - h_i))$, where (w_f, h_f) denote the width and height of free space f , and (w_i, h_i) represent the width and height of item i .

4. COMPUTATIONAL EXPERIMENTS 15

As a baseline, we select the best result obtained from all the combinations of sorting and placement heuristics performed separately for each problem type and size. This serves as an oracle-based benchmark, assuming an ideal scenario in which the best heuristic combination is known in advance for each dataset.

Due to the lack of standard benchmarks for 2D strip packing with limited item types, we also implemented the CP model. We used the OR-Tools CP-SAT solver, implemented as a Python package. Although the performance of the CP solver in this kind of 2D packing problem is not particularly useful for smaller problem sizes, the solver can reach solutions that are proved to be optimal within the assigned time limit.

The proposed constructive heuristic algorithm was implemented in C++. For black-box optimization, we utilized the CMA-ES library from the Laboratory for Computer Science, Université Paris-Sud (<https://github.com/CMA-ES/libcmaes>), authored by Emmanuel Benazera and supported by Nikolaus Hansen, a co-author of the original method. The neural network implementation leveraged Eigen, a C++ template library for linear algebra (<http://eigen.tuxfamily.org>). Unless otherwise mentioned, all experiments were conducted on a PC-class computer equipped with a 12-core processor, operating at an average clock speed of 4.3 GHz.

Based on our preliminary tests, we configured the population size to 192 and the initial step-size parameter σ to 0.4. Weights optimization in the learning phase terminates after 300,000 black-box function evaluations, equivalent to 1,560 iterations. We employed the Sep-CMA-ES variant, which achieves linear computational complexity with respect to the number of dimensions by restricting the covariance matrix to a diagonal form [32].

4.3 Results

Table 4 presents the results for the following algorithms:

- MaxRects and Skyline heuristics (MR) – reporting the best result among all variants for each problem size and type,
- Constraint Programming (CP) – with a 300-second time limit,
- The proposed single-pass neural-based heuristic (CH),
- The refined version using local search (RCH) – evaluated with a 30-second time limit.

Results reported in Table 4 represent the percentage of wasted space, the average over 10 randomly generated instances for the 2DCPackGen and logistic datasets. Wasted space is defined as the difference between the used strip area and the total area of the packed items, where the used strip area is computed as the product of the strip's width and the minimum height achieved by the algorithm.

The complexity of the problem increases with both the number of item types and the number of items per type. Additionally, instances involving mixed item sizes introduce further difficulty, often leading to poorer performance across

16 M. Kaleta et al.

methods. The constraint programming (CP) approach achieves superior results compared to MaxRects and Skyline (MR) heuristics for smaller and simpler instances. However, for larger cases, such as those with 15 item types and up to 20 items per type, CP begins to underperform relative to even basic heuristics. This trend is particularly noticeable in instances with mixed item sizes, where CP often fails to maintain its advantage. It is also important to note that MR, being a single-pass heuristic, completes its operation within milliseconds, which is significantly faster than the 300-second time limit of CP.

The neural-based constructive heuristic (CH) consistently outperforms the best results obtained across all MaxRects and Skyline (MR) heuristic variants. However, there are two exceptions where CH performs slightly worse than MR: (i) for 2DCPackGen instances with $|T| = 5$ and 1–10 items per type, and (ii) for logistic datasets with $|T| = 5$ and 1–20 items. This can be explained by the fact that the small-size instances are highly combinatorial, with the final result depending strongly on the top envelope of the strip, and it is possible that, just by chance, one of the MR alternative approaches is better in this specific instance set.

Similarly to MR, CH is less effective than constraint programming (CP) for smaller instances. Nevertheless, it demonstrates clear superiority as the number of item types, items per type, and item heterogeneity increase. This highlights the scalability of CH, especially considering it operates as a fast, single-pass algorithm, significantly outperforming CP in terms of runtime.

While CH already shows considerable promise, the integration of a local search component in the refined version (RCH) leads to substantial performance gains. With the exception of the smallest instances, CP no longer remains competitive. Even for moderate-sized problems—e.g., $|T| = 5$ with 1–10 items—the advantage of RCH is already apparent. For the largest instances, RCH improves over CH by up to a factor of two (e.g., 1.17% vs. 2.91%), and surpasses CP by an order of magnitude (e.g., 1.17% vs. 10.30%).

From a practical standpoint, RCH demonstrates even greater advantages on logistic datasets, especially as the number of item types and item counts increases. In these cases, the average wasted space is often reduced to below 1%, underscoring the effectiveness and real-world applicability of the proposed approach.

Figures 6 and 7 illustrate exemplary packing. While in the case of 2DCPackGen, there are a few areas of wasted space close to the right side of the strip and in the middle of the strip, packing for logistic instances typically fills the whole width of the strip, even if the pattern must be composed of different items. In general, packing looks tight, and it is hard to pinpoint any improvement manually.

To evaluate the convergence behavior of the proposed heuristic refinement algorithm, Table 5 presents the results obtained under varying time limits: 1, 3, 10, and 30 seconds. For comparison, we also include the results from the constraint programming (CP) solver using a 300-second time limit. In the case of the logistic datasets, only the 1-second and 30-second results are shown, as

4. COMPUTATIONAL EXPERIMENTS 17



Fig. 6. Example packing for three 2DCPackGen problem instances, mixed item sizes, 10 item types, and 1-5 items per type

the algorithm consistently reached its best solution within the first second. By contrast, convergence on the synthetic 2DCPackGen datasets is slightly slower. For smaller instances, performance typically stabilizes within 1 to 3 seconds, while for the largest and most complex problems, improvements continue up to the 10–30 second range.

18 M. Kaleta et al.

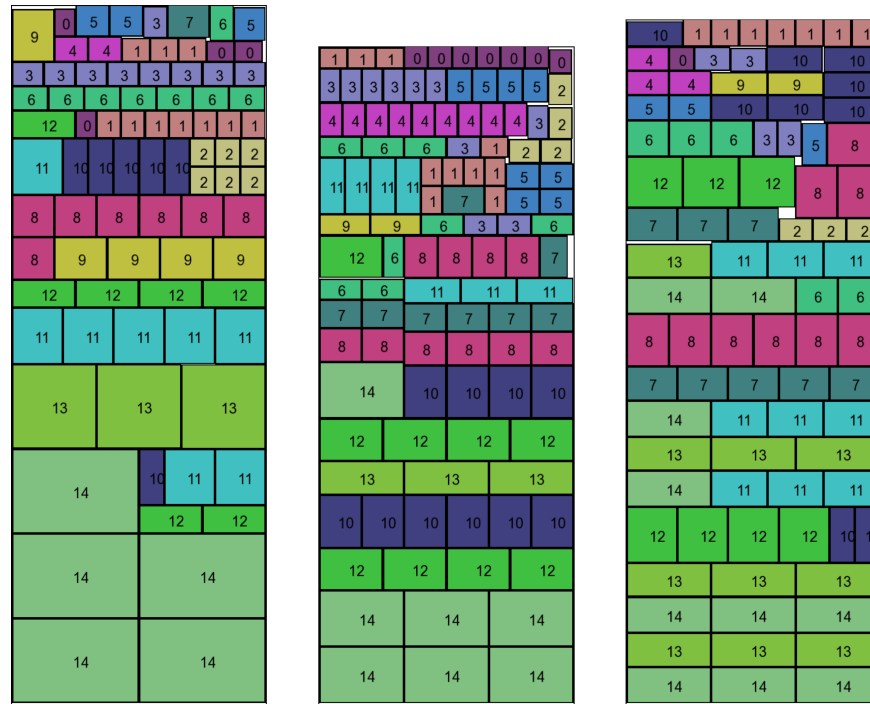


Fig. 7. Example packing for three logistic problem instances, mixed item sizes, 15 item types, and 1-10 items per type

5 Conclusions

In this paper, we proposed a novel approach to solving the two-dimensional strip packing problem (2D-SPP) by leveraging neural-driven constructive heuristics combined with local search in the space of heuristics rather than solutions. Our method builds on previous work by enhancing the adaptability of the neural-based framework through a black-box optimization process using CMA-ES, both during general training and problem-specific refinement.

Our results show that the proposed neural constructive heuristic (CH) consistently outperforms traditional single-pass heuristic approaches like MaxRects and Skyline. More notably, the refined version (RCH), enhanced via a local search in the heuristic space, delivers substantial improvements, achieving lower wasted space percentages across all tested datasets, especially in real-world-inspired logistics scenarios. In many cases, RCH even surpasses constraint programming (CP) methods in both efficiency and scalability, particularly for larger and more heterogeneous instances.

The experimental evidence confirms that learning and fine-tuning heuristics dynamically can lead to more efficient and generalizable solutions for NP-hard

5. CONCLUSIONS 19

combinatorial problems like 2D-SPP. Moreover, by shifting the optimization focus from solutions to heuristics, our approach opens up new possibilities for integrating domain knowledge, data-driven learning, and problem-specific customization.

Future research may explore extending this meta-heuristic framework to other combinatorial optimization problems, further refining the neural architecture, or integrating reinforcement learning components to enhance real-time adaptability.

20 M. Kaleta et al.

Table 4. Average wasted space [%].

T	d_t	2DCPackGen					logistic				
		ID	MR	CP	CH	RCH	Dim.	MR	CP	CH	RCH
5	1-5	1	14.69	3.62	12.09	4.92	Small	19.33	13.41	17.09	13.70
		6	18.08	9.57	13.33	10.40	Medium	14.88	5.36	13.10	5.64
		2	15.28	2.77	11.42	3.66	Large	10.38	2.60	8.71	3.73
		16	12.21	2.83	8.78	3.28	Mixed	13.82	4.63	12.35	5.58
	1-10	1	12.36	3.44	7.15	1.90	Small	16.97	5.17	12.33	4.47
		6	15.15	8.40	11.05	8.42	Medium	9.46	3.13	7.94	1.81
		2	10.81	2.98	7.11	2.34	Large	7.64	1.95	5.98	1.19
		16	8.14	3.35	8.32	2.56	Mixed	8.43	3.68	7.64	2.16
	1-20	1	9.09	4.53	6.25	1.67	Small	11.14	4.90	6.57	2.13
		6	14.45	8.44	10.2	8.09	Medium	4.92	4.50	4.48	0.90
		2	9.25	4.57	4.94	1.73	Large	4.00	3.54	2.92	0.58
		16	6.86	4.84	5.55	1.75	Mixed	4.60	5.02	4.85	0.95
10	1-5	1	11.32	3.21	6.61	2.26	Small	12.40	5.11	10.77	4.11
		6	14.21	4.78	7.68	4.82	Medium	10.28	3.83	6.56	1.48
		2	10.10	3.37	7.24	2.12	Large	6.55	2.47	4.65	0.75
		16	7.26	3.13	5.63	1.97	Mixed	8.44	3.37	8.08	1.74
	1-10	1	8.96	4.79	4.75	1.68	Small	10.64	5.25	6.68	1.60
		6	13.19	6.16	6.2	4.12	Medium	5.60	4.73	3.71	0.71
		2	7.78	5.06	5.38	1.67	Large	3.35	3.61	2.27	0.41
		16	5.70	4.63	4.45	1.64	Mixed	5.90	4.89	3.84	0.93
	1-20	1	6.74	6.38	3.76	1.35	Small	6.98	5.14	3.34	0.50
		6	12.62	8.54	5.92	3.86	Medium	3.46	6.30	2.02	0.36
		2	6.51	7.64	3.45	1.29	Large	2.54	5.66	1.49	0.19
		16	4.57	6.46	2.94	1.41	Mixed	4.20	6.58	2.19	0.41
15	1-5	1	10.26	4.01	6.27	2.18	Small	12.13	3.95	7.56	2.03
		6	13.22	6.83	9.02	5.94	Medium	9.10	4.55	4.37	1.17
		2	8.44	3.83	5.46	1.95	Large	4.66	3.40	3.20	0.59
		16	5.96	3.35	4.78	1.73	Mixed	6.62	3.95	4.89	0.97
	1-10	1	7.78	5.74	4.70	1.47	Small	10.30	4.81	5.20	1.11
		6	12.90	9.22	7.42	5.40	Medium	5.38	5.73	2.94	0.60
		2	7.37	6.46	3.84	1.36	Large	2.70	5.49	1.49	0.24
		16	5.02	5.41	3.11	1.37	Mixed	5.72	5.64	2.35	0.57
	1-20	1	5.68	10.30	2.91	1.17	Small	6.68	7.97	2.63	0.35
		6	12.37	11.03	6.44	4.95	Medium	2.69	8.58	1.47	0.29
		2	6.50	10.68	2.47	1.04	Large	1.84	7.50	0.91	0.14
		16	4.35	9.26	2.32	1.18	Mixed	3.49	9.25	1.30	0.21

5. CONCLUSIONS 21

Table 5. Average wasted space vs. computation time for RCH algorithm [%].

T	d_t	2DCPackGen						logistic			
		ID	CP	1s.	3s.	10s.	30s.	Dim.	CP	1s.	30s.
5	1-5	1	3.62	4.95	4.92	4.92	4.92	Small	13.41	13.70	13.70
		6	9.57	10.40	10.40	10.40	10.40	Medium	5.36	5.64	5.64
		2	2.77	3.66	3.66	3.66	3.66	Large	2.60	3.73	3.73
		16	2.83	3.28	3.28	3.28	3.28	Mixed	4.63	5.58	5.58
	1-10	1	3.44	1.93	1.90	1.90	1.90	Small	5.17	4.47	4.47
		6	8.40	8.42	8.42	8.42	8.42	Medium	3.13	1.81	1.81
		2	2.98	2.56	2.34	2.34	2.34	Large	1.95	1.19	1.19
		16	3.35	2.61	2.56	2.56	2.56	Mixed	3.68	2.16	2.16
	1-20	1	4.53	1.84	1.68	1.67	1.67	Small	4.90	2.13	2.13
		6	8.44	8.10	8.09	8.09	8.09	Medium	4.50	0.90	0.90
		2	4.57	1.86	1.73	1.73	1.73	Large	3.54	0.58	0.58
		16	4.84	1.81	1.75	1.75	1.75	Mixed	5.02	0.95	0.95
10	1-5	1	3.21	2.50	2.26	2.26	2.26	Small	5.11	4.11	4.11
		6	4.78	4.93	4.82	4.82	4.82	Medium	3.83	1.48	1.48
		2	3.37	2.32	2.12	2.12	2.12	Large	2.47	0.75	0.75
		16	3.13	2.00	1.98	1.97	1.97	Mixed	3.37	1.74	1.74
	1-10	1	4.79	2.07	1.76	1.68	1.68	Small	5.25	1.60	1.60
		6	6.16	4.38	4.18	4.12	4.12	Medium	4.73	0.71	0.71
		2	5.06	2.05	1.80	1.67	1.67	Large	3.61	0.41	0.41
		16	4.63	1.82	1.69	1.64	1.64	Mixed	4.89	0.93	0.93
	1-20	1	6.38	1.95	1.72	1.39	1.35	Small	5.14	0.50	0.50
		6	8.54	4.20	4.02	3.86	3.86	Medium	6.30	0.36	0.36
		2	7.64	1.92	1.72	1.30	1.29	Large	5.66	0.19	0.19
		16	6.46	1.65	1.52	1.42	1.41	Mixed	6.58	0.41	0.41
15	1-5	1	4.01	2.67	2.35	2.18	2.18	Small	3.95	2.03	2.03
		6	6.83	6.29	6.05	5.94	5.94	Medium	4.55	1.17	1.17
		2	3.83	2.65	2.15	1.95	1.95	Large	3.40	0.59	0.59
		16	3.35	1.97	1.76	1.73	1.73	Mixed	3.95	0.97	0.97
	1-10	1	5.74	2.10	1.87	1.47	1.47	Small	4.81	1.11	1.11
		6	9.22	5.79	5.56	5.40	5.40	Medium	5.73	0.60	0.60
		2	6.46	2.07	1.85	1.42	1.36	Large	5.49	0.24	0.24
		16	5.41	1.82	1.56	1.37	1.37	Mixed	5.64	0.57	0.57
	1-20	1	10.30	1.89	1.83	1.50	1.17	Small	7.97	0.35	0.35
		6	11.03	5.25	5.16	5.00	4.95	Medium	8.58	0.29	0.29
		2	10.68	1.66	1.60	1.27	1.04	Large	7.50	0.14	0.14
		16	9.26	1.64	1.48	1.26	1.18	Mixed	9.25	0.21	0.21

22 M. Kaleta et al.

References

1. A. Neuenfeldt Júnior, E. Silva, M. Francescatto, C. B. Rosa, and J. Siluk, "The rectangular two-dimensional strip packing problem real-life practical constraints: A bibliometric overview," *Computers & Operations Research*, vol. 137, p. 105521, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0305054821002616>
2. A. Lodi, S. Martello, and M. Monaci, "Two-dimensional packing problems: A survey," *European Journal of Operational Research*, vol. 141, no. 2, pp. 241–252, 2002. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221702001236>
3. M. Kaleta and T. Śliwiński, "Neural-driven heuristic for strip packing trained with black-box optimization," *International Journal of Electronics and Telecommunications*, vol. 71, no. 2, 2025.
4. X. Zhao, Y. Rao, and J. Fang, "A reinforcement learning algorithm for the 2d-rectangular strip packing problem," *Journal of Physics: Conference Series*, vol. 2181, no. 1, p. 012002, jan 2022. [Online]. Available: <https://dx.doi.org/10.1088/1742-6596/2181/1/012002>
5. J. Oliveira, A. Neuenfeldt Júnior, E. Silva, and M. Carravilla, "A survey on heuristics for the two-dimensional rectangular strip packing problem," *Pesquisa Operacional*, vol. 36, pp. 197–226, 08 2016.
6. B. Baker, E. Coffman, and R. Rivest, "Orthogonal packings in two dimensions," *SIAM J. Comput.*, vol. 9, pp. 846–855, 11 1980.
7. E. K. Burke, G. Kendall, and G. Whitwell, "A new placement heuristic for the orthogonal stock-cutting problem," *Operations Research*, vol. 52, no. 4, pp. 655–671, 2004. [Online]. Available: <http://www.jstor.org/stable/30036614>
8. J. Jylänki, "A thousand ways to pack the bin-a practical approach to two-dimensional rectangle bin packing," *retrived from http://clb. demon.fi/files/RectangleBinPack. pdf*, 2010.
9. L. Wei, A. Lim, and W. Zhu, "A skyline-based heuristic for the 2d rectangular strip packing problem," in *Modern Approaches in Applied Intelligence*, K. G. Mehrotra, C. K. Mohan, J. C. Oh, P. K. Varshney, and M. Ali, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 286–295.
10. S. Martello, D. Pisinger, and D. Vigo, "The three-dimensional bin packing problem," *Operations Research*, vol. 48, no. 2, pp. 256–267, 2000. [Online]. Available: <http://www.jstor.org/stable/223143>
11. T. G. Crainic, G. Perboli, and R. Tadei, "Extreme point-based heuristics for three-dimensional bin packing," *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008. [Online]. Available: <https://doi.org/10.1287/ijoc.1070.0250>
12. J. R. Rice, "The algorithm selection problem," ser. *Advances in Computers*, M. Rubinfeld and M. C. Yovits, Eds. Elsevier, 1976, vol. 15, pp. 65–118. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0065245808605203>
13. J. Huang, "Optimal linear combination of heuristic strategies for 2d rectangular bin packing algorithms," in *2024 IEEE 2nd International Conference on Image Processing and Computer Applications (ICIPCA)*, 2024, pp. 1867–1876.
14. R. Rakotonirainy, "A machine learning approach for automated strip packing algorithm selection," *ORiON*, vol. 36, pp. 73–, 02 2021.
15. M. Beyaz, T. Dokeroglu, and A. Cosar, "Robust hyper-heuristic algorithms for the offline oriented/non-oriented 2d bin packing problems," *Applied Soft Computing*, vol. 36, pp. 236–245, 2015.

5. CONCLUSIONS 23

16. K. Piechowiak, M. Drozdowski, and Éric Sanlaville, "Framework of algorithm portfolios for strip packing problem," *Computers Industrial Engineering*, vol. 172, p. 108538, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360835222005460>
17. W. Kolodziejczyk and M. Kaleta, "Mitigating Dimensionality in 2D Rectangle Packing Problem under Reinforcement Learning Schema," in *Progress in Polish Artificial Intelligence Research 5*, J. Mańdziuk, A. Żychowski, and M. Małkiński, Eds. Warsaw University of Technology, 2024, pp. 412–418. [Online]. Available: <https://doi.org/10.17388/WUT.2024.0002.MiN1>
18. J. Fan, "A review for deep reinforcement learning in atari: Benchmarks, challenges and solutions," EasyChair Preprint 6985, EasyChair, 2023.
19. J. Fang, Y. Rao, and M. Shi, "A deep reinforcement learning algorithm for the rectangular strip packing problem," *PLOS ONE*, vol. 18, no. 3, pp. 1–20, 03 2023. [Online]. Available: <https://doi.org/10.1371/journal.pone.0282598>
20. Y. Xu and Z. Yang, "Graphpack: A reinforcement learning algorithm for strip packing problem using graph neural network," *Journal of Circuits, Systems and Computers*, vol. 33, no. 08, p. 2450139, 2024. [Online]. Available: <https://doi.org/10.1142/S0218126624501391>
21. K. Zhu, N. Ji, and X. D. Li, "Hybrid heuristic algorithm based on improved rules & reinforcement learning for 2d strip packing problem," *IEEE Access*, vol. 8, pp. 226 784–226 796, 2020.
22. A. Neuenfeldt Júnior, J. Siluk, M. Francescatto, G. Stieler, and D. Disconzi, "A framework to select heuristics for the rectangular two-dimensional strip packing problem," *Expert Systems with Applications*, vol. 213, p. 119202, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417422022205>
23. J. Arabas and D. Jagodziński, "Toward a matrix-free covariance matrix adaptation evolution strategy," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 1, pp. 84–98, 2020.
24. P. Carvalho, N. Lourenço, and P. Machado, "How to improve neural network training using evolutionary algorithms," *SN Computer Science*, vol. 5, no. 6, p. 664, Jun 2024. [Online]. Available: <https://doi.org/10.1007/s42979-024-02972-5>
25. D. Jagodziński, L. Neumann, and P. Zawistowski, "Deep neuroevolution: Training neural networks using a matrix-free evolution strategy," in *Neural Information Processing*, T. Mantoro, M. Lee, M. A. Ayu, K. W. Wong, and A. N. Hidayanto, Eds. Cham: Springer International Publishing, 2021, pp. 524–536.
26. N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
27. I. Sabuncuoglu and M. Bayiz, "Job shop scheduling with beam search," *European Journal of Operational Research*, vol. 118, no. 2, pp. 390–412, 1999. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221798003191>
28. S. Wang, Z. Bing-Hai, and X. Li-Feng, "A filtered-beam-search-based heuristic algorithm for flexible job-shop scheduling problem," *International Journal of Production Research - INT J PROD RES*, vol. 46, pp. 3027–3058, 06 2008.
29. E. Birgin, J. Ferreira, and D. Ronconi, "List scheduling and beam search methods for the flexible job shop scheduling problem with sequencing flexibility," *European Journal of Operational Research*, vol. 247, no. 2, pp. 421–440, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221715005378>
30. E. Silva, J. F. Oliveira, and G. Wäscher, "2dcpackgen: A problem generator for two-dimensional rectangular cutting and packing problems," *European Journal of Operational Research*, vol. 237, no. 3, pp. 846–856, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221714002112>

24 M. Kaleta et al.

31. G. Wäscher, H. Haußner, and H. Schumann, “An improved typology of cutting and packing problems,” *European Journal of Operational Research*, vol. 183, no. 3, pp. 1109–1130, 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S037722170600292X>
32. R. Ros and N. Hansen, “A simple modification in cma-es achieving linear time and space complexity,” in *Parallel Problem Solving from Nature – PPSN X*, G. Rudolph, T. Jansen, N. Beume, S. Lucas, and C. Poloni, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 296–305.